

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

2026-09-20

Rules, judge, loop

W4

BLOCK 1 · CLASS 4 · PRACTICAL

Rules, judge, loop.

Last week you watched a feature get built. Today you build one.

Timing: open + check-in 5 · what an agent is 17 · three standards 13 · git standards 5 · live kick-off 10 · your turn 30 · retro 6 · wrap 2 = 88, plus the two dividers. TA-run. Slides carry ideas only: every command is on the Week 4 page, open on their laptops, and in these notes. Presenting laptop: SplitIt with feature 1 merged, clean main, Codex open, GitHub Desktop signed in, data reset.

Today.

- 1 Check-in
- 2 What an agent actually is
- 3 Three standards
- 4 Git: one change, start to finish
- 5 Build one, live — then you build one
- 6 Retro: one new rule

2026-09-20

Rules, judge, loop

└ Today.

Today.

- 1 Check-in
- 2 What an agent actually is
- 3 Three standards
- 4 Git: one change, start to finish
- 5 Build one, live — then you build one
- 6 Retro: one new rule

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

2026-09-20

Rules, judge, loop

Show me your GitHub page.

Walk the room looking at browsers, not code. Two minutes per row; note who is behind. They can pull main and run the feature-1 tests while you walk — command on the Week 4 page. Don't debug git at the front: point at last week's "when git is confusing" slide.

CHECK-IN · 5 MIN

Show me your GitHub page.

- A branch for feature 1.
- A merged pull request.

Not there yet? Start there today.

WHAT IS AN AGENT?

A chat model writes. An agent **acts**.

A CHAT MODEL

Text in, text out.

It has never seen your folder.

AN AGENT

The same model, with hands.

Reads. Edits. Runs. Sees the result.
Chooses again.

Agent = model + tools + a loop.

2026-09-20

Rules, judge, loop

└ A chat model **writes**. An agent **acts**.

One minute, from their own experience: in Class 2 they asked Codex to show a line of a data file and it showed the real line — a browser chat could not have. Same model family, different powers. "Agentic" just means: can act, not only answer.

WHAT IS AN AGENT?

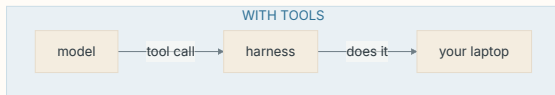
A chat model writes. An agent **acts.**

A CHAT MODEL
Text in, text out.
It has never seen your folder.

AN AGENT
The same model, with hands.
Reads. Edits. Runs. Sees the result.
Chooses again.

Agent = model + tools + a loop.

Without tools it can only give advice.



WHAT A TOOL IS

A named action the model is allowed to ask for. Read a file · edit a file · run a command.

WHY IT CHANGES EVERYTHING

The result comes back *to the model*. Without tools you are the hands — and the only one who finds out.

2026-09-20

Rules, judge, loop

Without tools it can only give **advice**.

The top lane is what they did all through Class 2 with a browser chat: the model advises, they carry it out, they report back. They were the tool. The bottom lane is Codex. Ask the room: "in the top lane, who finds out if the code was wrong?" — they do, eventually. In the bottom lane the model finds out immediately, which is the whole reason a loop is possible. Three tools cover almost everything; the next slide is the protocol.

WHAT IS AN AGENT? · TOOLS

Without tools it can only give advice.



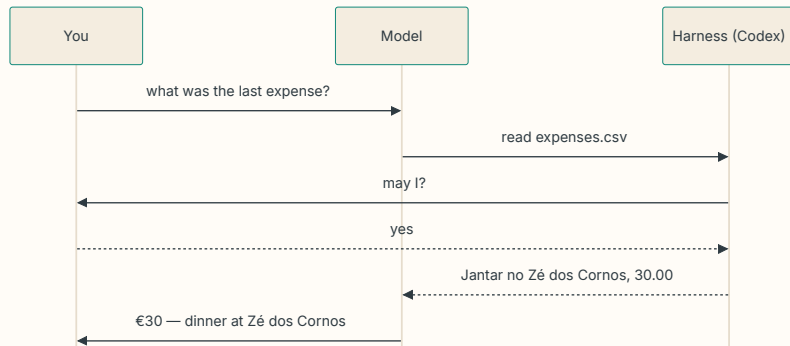
WHAT A TOOL IS

A named action the model is allowed to ask for. Read a file · edit a file · run a command.

WHY IT CHANGES EVERYTHING

The result comes back to the model. Without tools you are the hands — and the only one who finds out.

The model asks. The harness does.

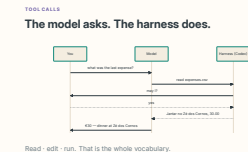


Read · edit · run. That is the whole vocabulary.

2026-09-20

Rules, judge, loop

└─ The model asks. The harness does.



This is the exact thing they did in Class 2 — they asked Codex about a line of their own expenses file and it came back with the real line. Walk it left to right: a plain question, a precise request, the permission gate, the file's own words, a plain answer. Two things to point at. The model never saw the file — it had to ask. And the approval prompt is where you sit inside the loop; that is the only place a human is required. Later, narrate the live kick-off in the same words — "that's a read, that's an edit, that's a run" — so the diagram and the demo are one thing.

What actually goes back and forth.

YOU TYPE	what was the last expense?
THE MODEL EMITS	<code>{"tool": "read_file", "path": "data/expenses.csv"}</code>
CODEX ASKS	Codex wants to read <code>data/expenses.csv</code> . Allow?
CODEX PASTES BACK	<code>29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25</code>
THE MODEL ANSWERS	€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked; you allowed it; the file's own line was pasted into the conversation. That line is the only reason it can answer.

2026-09-20

Rules, judge, loop

└─What actually goes back and forth.

Read it top to bottom, slowly — this is the slide that demystifies the whole thing. Three things to point at. (1) The model's output is not prose: it is a small structured request naming a tool and what to use it on. It cannot open anything; it can only ask. (2) Codex — the harness — is what actually touches the disk, and it stops to ask you first; that prompt is the same one they click through all lesson. (3) The file's real line is pasted into the conversation as ordinary text, and from there the model reads it like any other message. Ask the room: "if we had said no, what could it have answered?" Nothing — it would have to guess. The JSON is illustrative; each harness has its own wire format, and they never type it themselves.

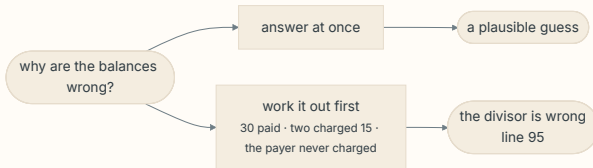
TOOL CALLS · ONE EXCHANGE, IN FULL

What actually goes back and forth.

YOU TYPE	what was the last expense?
THE MODEL EMITS	<code>{"tool": "read_file", "path": "data/expenses.csv"}</code>
CODEX ASKS	Codex wants to read <code>data/expenses.csv</code> . Allow?
CODEX PASTES BACK	<code>29,3,8,Jantar no Zé dos Cornos,30.00,2026-08-25</code>
THE MODEL ANSWERS	€30 — dinner at Zé dos Cornos, on 25 August.

It never saw the file. It asked; you allowed it; the file's own line was pasted into the conversation. That line is the only reason it can answer.

Answer at once — or **work it out**.



WHAT IT IS

Work it out in steps, *then* decide — as you would on paper.

THE PROBLEM IT SOLVES

A model answers in one pass — the same effort for $2 + 2$ as for your bug. The steps it writes **are** the extra thinking.

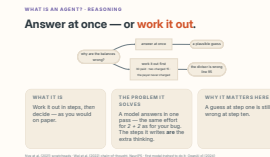
WHY IT MATTERS HERE

A guess at step one is still wrong at step ten.

2026-09-20

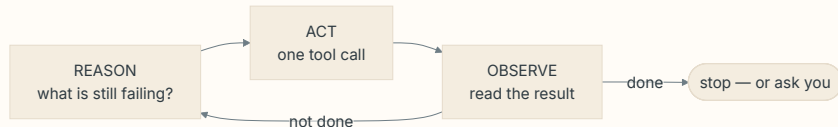
Rules, judge, loop

Answer at once — or **work it out**.



Use their own bug: this is SplitIt's balances from Class 2. The deep reason, if anyone asks: a transformer spends a fixed amount of computation per token, so answering straight away means committing with one fixed budget however hard the question is. Writing intermediate steps buys more computation and gives the model a scratchpad it can read back — which is why "think step by step" is not a politeness trick. Lineage: Nye et al. 2021 called it a scratchpad; Wei et al. 2022 named it chain-of-thought and showed it emerges in large models; Kojima et al. 2022 found "let's think step by step" alone does much of it; OpenAI's o1 (Sept 2024) was the first model trained by reinforcement learning to do it unprompted, followed by DeepSeek-R1 and Claude's extended thinking. Say the honest part too — a guess and a worked answer look equally confident on screen, which is exactly why you still read the diff.

The loop has a name: **ReAct**.



THE HARNESS

Runs the tool. Asks you. Feeds the result back. Decides when to stop. Codex is one; so are Claude Code and Cursor.

WHY YOU CARE

They all run this same loop. Learn it once, and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

2026-09-20

Rules, judge, loop

└ The loop has a name: **ReAct**.

REASON · ACT · OBSERVE

The loop has a name: **ReAct**.

THE HARNESS

Runs the tool. Asks you. Feeds the result back. Decides when to stop. Codex is one; so are Claude Code and Cursor.

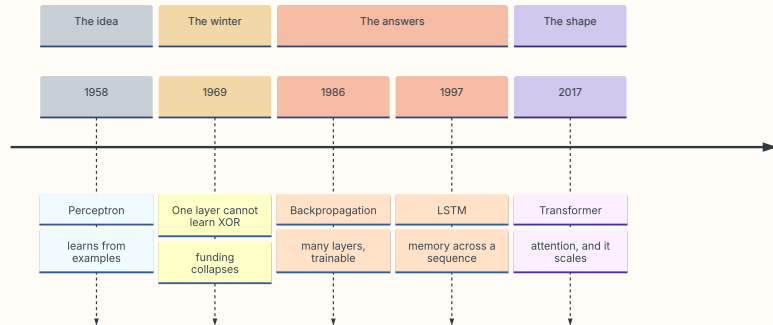
WHY YOU CARE

They all run this same loop. Learn it once, and the tools are interchangeable.

Yao, Zhao, Yu, Du, Shafraan, Narasimhan & Cao (2022). *ReAct: Synergizing Reasoning and Acting in Language Models*. ICLR 2023 · arxiv.org/abs/2210.03629

ReAct = "Reasoning and Acting", Yao et al. 2022 (ICLR 2023, arXiv:2210.03629): a model does far better alternating a thought with an action and reading the result than answering in one go. Every harness since is that loop with better plumbing. The line to land: the three standards are you configuring the harness — the rulebook it loads, the judge it runs, the stop condition.

The ideas came first. By decades.

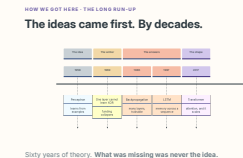


Sixty years of theory. **What was missing was never the idea.**

2026-09-20

Rules, judge, loop

└ The ideas came first. By decades.



Ninety seconds, left to right. The dates are on the slide; the names and the story are here. 1958, Rosenblatt's perceptron: a machine that adjusts its own weights from examples instead of being programmed rule by rule. That is still the whole idea. 1969, Minsky and Papert's book *Perceptrons*: one layer can only separate what a straight line separates, so it cannot learn XOR — about as simple as a function gets. Funding collapsed; the first AI winter. Careful with the attribution: Minsky did not invent multi-layer networks. Everyone knew more layers would fix XOR; nobody could train them — his critique is what made that the open problem. 1986, Rumelhart, Hinton and Williams in *Nature*: backpropagation sends the error back through every layer, so many layers finally become trainable. The answer to 1969, seventeen years late. 1997, Hochreiter and Schmidhuber: LSTM carries memory across a sequence. 2017, Vaswani et al.: the Transformer drops recurrence for attention and trains in parallel — every model two slides from now is one of these. If it comes up: Hinton shared the 2024 Nobel Prize in Physics with John Hopfield for this line of work.

Compute and text arrived **at the same time**.

COMPUTE

Arithmetic got cheap

2007 — graphics cards became programmable. 2012 — a network trained on two of them won the image contest outright.

DATA

The web became a corpus

From 2007 — a non-profit crawling the public web and giving it away. Billions of pages of text, free.

Neural nets are not new. **Cheap parallel arithmetic and a scraped web are.**

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (founded 2007, public crawls from 2011)

2026-09-20

Rules, judge, loop

└─ Compute and text arrived **at the same time**.

This is the payoff of the previous slide, and the line to land: the ideas sat there for decades because the two things they needed had not arrived. **Compute** — NVIDIA's CUDA (2007) let ordinary programmers run their own maths on a graphics card, which is thousands of tiny arithmetic units built for game pixels and, as it turns out, exactly what a neural network needs. In 2012 AlexNet — Krizhevsky, Sutskever and Hinton — trained on two consumer GPUs and won the ImageNet competition with roughly half the error of the runner-up; that is the moment it became obvious to everyone. **Data** — Common Crawl, a non-profit founded in 2007 publishing full-scale crawls from 2011: the web itself, free, as a training corpus. Neither one is an idea about intelligence. If someone asks why this happened now and not in 1990, that is the whole answer.

HOW WE GOT HERE · WHY IT HAPPENED NOW

Compute and text arrived **at the same time**.

COMPUTE

Arithmetic got cheap

2007 — graphics cards became programmable. 2012 — a network trained on two of them won the image contest outright.

DATA

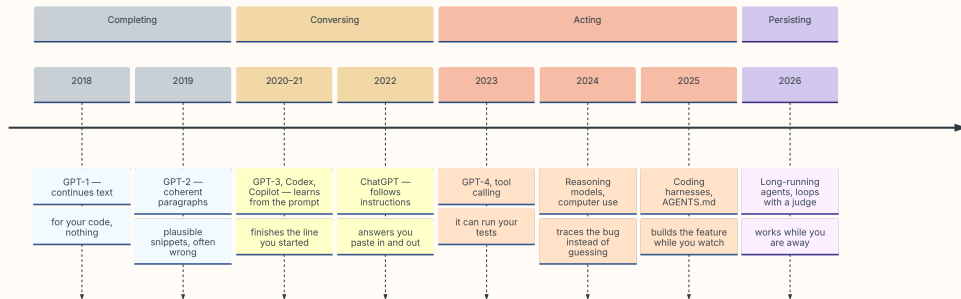
The web became a corpus

From 2007 — a non-profit crawling the public web and giving it away. Billions of pages of text, free.

Neural nets are not new. Cheap parallel arithmetic and a scraped web are.

NVIDIA CUDA (2007) · Krizhevsky, Sutskever & Hinton, AlexNet (2012) · Common Crawl (founded 2007, public crawls from 2011)

Then eight years, four kinds of capability.



The job changed twice: when it learned to take **instructions**, and when it got **tools and a loop**.

2026-09-20

Rules, judge, loop

Then eight years, four kinds of capability.

Two minutes, one sweep left to right. Sixty years fitted onto the last two slides; these eight get their own because this is where the job changed, not just the size. Say the four band names as you go — completing, conversing, acting, persisting — that is the shape to remember, not the model names. If asked for an example of reasoning: an older model asked why the balances were wrong guesses a plausible edit; a reasoning model works the arithmetic out and lands on the actual line. Dates are approximate release months on purpose.

Then eight years, four kinds of capability.



The job changed twice: when it learned to take instructions, and when it got tools and a loop.

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

2026-09-20

Rules, judge, loop

3

THE CORE OF THE CLASS

Three standards.

A rulebook it reads. A judge it cannot argue with. A loop that knows when to stop.

AGENTS.md — read before anything else.

WHAT IT IS

A README for agents.

Every session starts by reading it.

WHAT GOES IN

What it cannot guess.

Commands. Boundaries. How to report back.

WHAT STAYS OUT

What it can read.

Your code, explained. Every extra line hides a real one.

It lives in git. When the agent misbehaves, fix a line here — not the prompt.

2026-09-20

Rules, judge, loop

AGENTS.md — read before anything else.

Ask Codex on stage: "What does AGENTS.md tell you to do?" — it recites the file. That is the proof it reads it. The test for every line: would removing it cause a mistake? If not, cut it. Mention the 32 KB cap only if asked; the real cap is attention.

STANDARD 1 · THE RULEBOOK

AGENTS.md — read before anything else.

WHAT IT IS
A README for
agents.
Every session starts by
reading it.

WHAT GOES IN
What it cannot
guess.
Commands. Boundaries.
How to report back.

WHAT STAYS OUT
What it can read.
Your code, explained.
Every extra line hides a
real one.

It lives in git. When the agent misbehaves, fix a line here — not the prompt.

Three rules. One commit.

BOUNDARY

One branch per spec.

Never straight to `main`.

EVIDENCE

Show me the test output.

Not a sentence saying it passed.

OFF-LIMITS

Never touch the tests.

If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

2026-09-20

Rules, judge, loop

└ Three rules. One commit.

Do it on stage: ask Codex to add the three rules under "How to answer" in AGENTS.md, read the diff aloud, accept, commit in GitHub Desktop. The third already existed in spirit — now it is explicit, and the student enforces it. Codex only reads AGENTS.md at the start of a thread, so say "new thread" now.

STANDARD 1 • LIVE • 4 MIN

Three rules. One commit.

BOUNDARY

One branch per spec.
Never straight to `main`.

EVIDENCE

Show me the test output.
Not a sentence saying it passed.

OFF-LIMITS

Never touch the tests.
If one looks wrong, say so and stop.

Exact wording on the Week 4 page. Add them, read the diff, commit, push.

Decide what **done** means. First.

IT STOPS WHEN IT LOOKS DONE

So give it a check.

Failing tests are the to-do list. Green is the finish line.

NEVER MOVE THE GOALPOSTS

Editing a test is cheating.

Not finishing.

EVIDENCE, NOT CLAIMS

Output, not adjectives.

"All tests pass" is a sentence.

GREEN IS NOT DONE

Read the diff. Click the app.

App wrong, tests green? You found a missing test.

2026-09-20

Rules, judge, loop

└─ Decide what **done** means. First.

Run the red tests on stage (command on the Week 4 page) and open the test file next to the spec: one test per checklist line. "Green is not done" is the card that pays off today — feature 2's form field is not tested at all, so only clicking the app catches a missing one.

STANDARD 2 · THE JUDGE

Decide what **done** means. First.

IT STOPS WHEN IT LOOKS DONE

So give it a check.

Failing tests are the to-do list. Green is the finish line.

NEVER MOVE THE GOALPOSTS

Editing a test is cheating.

Not finishing.

EVIDENCE, NOT CLAIMS

Output, not adjectives.

"All tests pass" is a sentence.

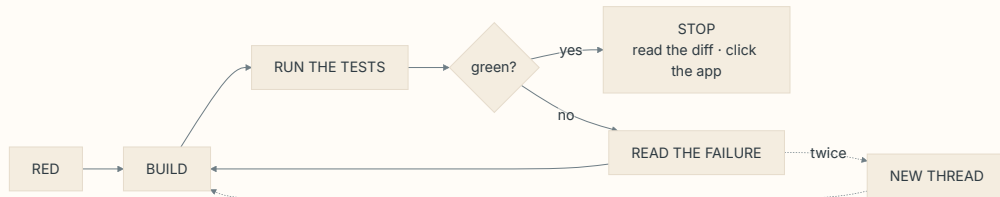
GREEN IS NOT DONE

Read the diff. Click the app.

App wrong, tests green? You found a missing test.

STANDARD 3 · THE LOOP

Red → build → run → fix → green.



ONE TASK PER LOOP

One bug, or one spec.

LET IT LOOP

It reads its own failures.
You read the result.

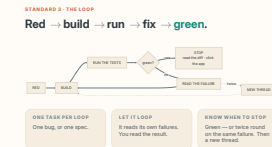
KNOW WHEN TO STOP

Green — or twice round
on the same failure. Then
a new thread.

2026-09-20

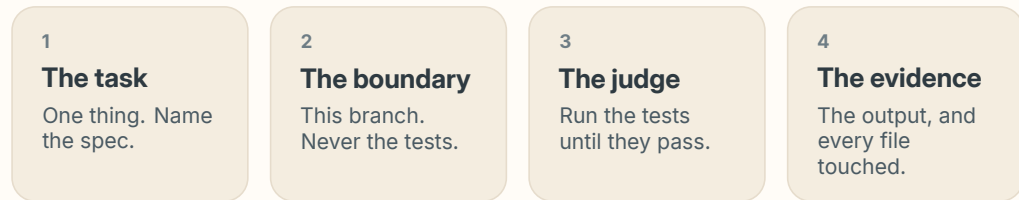
Rules, judge, loop

Red → build → run → fix → green.



The three parts are the whole lesson: a rulebook, a judge, a loop. Everything else in agentic coding is a variation. Today the loop runs inside one Codex turn and you watch; the same idea one level up is a script that re-feeds the prompt until the judge passes — Part II, Block 3.

Every good ask has four parts.



Written out in full on the Week 4 page. Copy it; change two names.

2026-09-20

Rules, judge, loop

└ Every good ask has four parts.

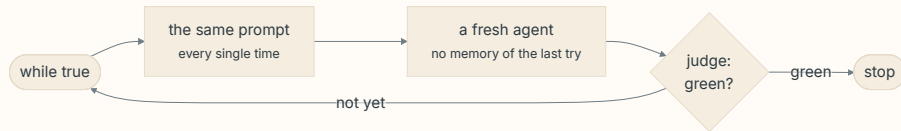
Four parts, in every loop prompt they will ever write. Make them name the four back to you before you paste it. New thread before pasting — the AGENTS.md rules from five minutes ago are only read at the start of a thread.

Every good ask has four parts.



Written out in full on the Week 4 page. Copy it; change two names.

Meet Ralph.



WHAT IT IS

The same prompt, handed to a fresh agent again and again, until the judge says green.

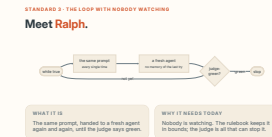
WHY IT NEEDS TODAY

Nobody is watching. The rulebook keeps it in bounds; the judge is all that can stop it.

2026-09-20

Rules, judge, loop

Meet Ralph.



Thirty seconds, as a teaser — you are not demoing this today. It is named after Ralph Wiggum: cheerfully, relentlessly wrong, and it gets there anyway. Geoffrey Huntley's original was literally 'while ;; do cat PROMPT.md | agent; done', and his line for why it works is "deterministically bad in an undeterministic world" — bounded, repeatable failure plus a check that cannot lie. The point to land, and the reason it closes this act: the agent has no memory between runs, so the PROMPT and the repo carry all the state. That is exactly why a rulebook and a judge are not housekeeping — they are the only things between you and an agent confidently going the wrong way for an hour. Part II, with the team project.



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

2026-09-20

Rules, judge, loop



HOW A TEAM STAYS SANE

One change, start to finish.

`main` is the version everyone trusts. Here is how a change gets into it.

Seven minutes for the whole story — the graph grows by one move per slide, so each takes about forty seconds. They met repo, commit, branch and push last week as words; this is the same four words as a single narrative, in the shape they will actually see in GitHub Desktop. Frame it the way they already think: a draft, checkpoints, a review, a sign-off.

This is `main`.



One version, one history, one truth — no folder full of *model_v3_FINAL_jm*.
Everybody's copy starts from here.

2026-09-20

Rules, judge, loop

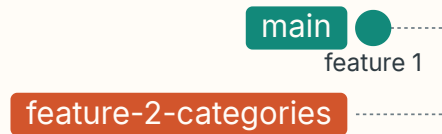
└─ This is `main`.

This is `main`.



One version, one history, one truth — no folder full of *model_v3_FINAL_jm*.
Everybody's copy starts from here.

You never edit the original.



A **branch** is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

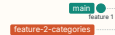
2026-09-20

Rules, judge, loop

└─ You never edit the original.

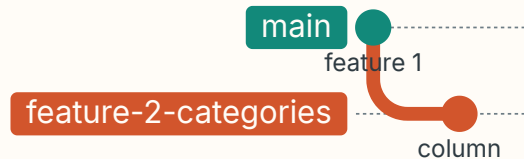
GIT · A BRANCH IS YOUR DRAFT

You never edit the original.



A branch is your own copy to work in, named after the job. Everyone else carries on from `main`, undisturbed.

Every save point is a commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

2026-09-20

Rules, judge, loop

└ Every save point is a commit.

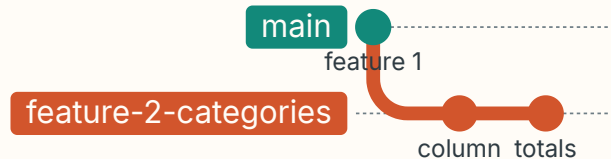
GIT · CHECKPOINTS YOU CAN EXPLAIN

Every save point is a commit.



A **commit** is a checkpoint with a note saying what changed. Small ones. You can go back to any of them.

`main` has not moved.



Three commits deep, half of it broken, the agent having a bad day — the version everyone else is using is exactly as you left it.

2026-09-20

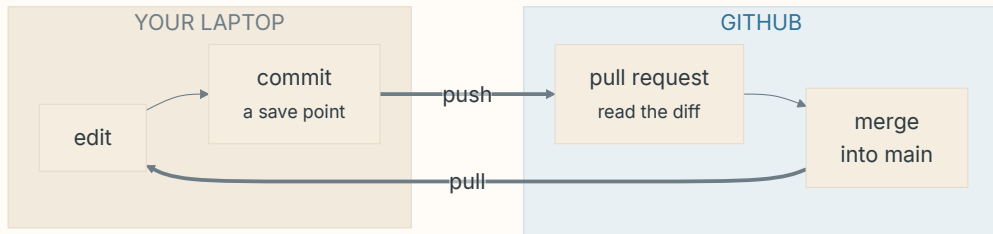
Rules, judge, loop

`main` has not moved.



Three commits deep, half of it broken, the agent having a bad day — the version everyone else is using is exactly as you left it.

Until you push, it is only on your laptop.



Push puts your draft where the team can see it. A **pull request** is you asking someone to read the diff before it counts — today, that someone is you, reviewing the agent.

2026-09-20

Rules, judge, loop

└─ Until you push, it is only on your laptop.

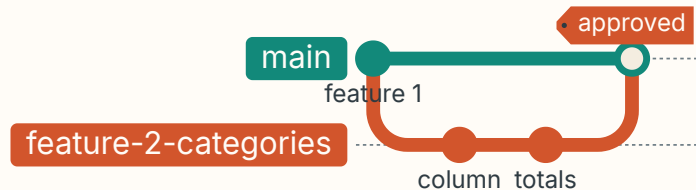
GIT · SENDING IT FOR REVIEW

Until you push, it is only on your laptop.



Push puts your draft where the team can see it. A pull request is you asking someone to read the diff before it counts — today, that someone is you, reviewing the agent.

Approved. Now it is the official version.



Merge is the sign-off. `main` now includes your work — and the next person's copy starts from here.

2026-09-20

Rules, judge, loop

Approved. Now it is the official version.

GIT • THE SIGN-OFF

Approved. Now it is the official version.



Merge is the sign-off. `main` now includes your work — and the next person's copy starts from here.

Land the cycle: they are now back at slide one, with a bigger main. That is the whole habit. Two rules fall out of the story rather than being announced — name the branch after the job (slide 2), and keep commits small enough to explain (slide 3). If someone asks what happens to the branch: it is finished, GitHub offers to delete it, say yes.

Watch it work. Then check its work.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

2026-09-20

Rules, judge, loop

└─ Watch it work. Then check its work.

Let it loop for real; 2-4 rounds is normal. Narrate it as tool calls. Commands and file names are on the Week 4 page if you want them on screen. The two things it commonly misses on this feature: it forgets to refresh the data files from the seed (tests stay green while the app crashes on load) and it never adds the form field (nothing tests that screen). Both are the "green is not done" lesson — show them, do not fix them silently. If it edits a test: stop, discard, point at the rule, re-prompt.

LIVE • FEATURE 2 • 10 MIN

Watch it work. Then check its work.

- 1 Branch. Named after the spec.
- 2 Red first. See what is missing before anything is written.
- 3 New thread. Paste the ask. Then sit on your hands.
- 4 Watch it loop.
- 5 Green — now don't trust it. Read the diff. Click the app.
- 6 Commit, push, pull request, merge.

One bug on `main`. One feature on a branch.

SPLITIT

Bug 2, then feature 2

Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2

"Won this month" is always zero. Then
conversion by source.

Files, test names and the ask: Week 4 page.

2026-09-20

Rules, judge, loop

└ One bug on `main`. One feature on a branch.

Roam for two things: did they ask for the test output, and did they read the diff before accepting. Never answer a failing test — "what does the last line say?". SplitIt bug 002: the test demands a helper that also removes the member's expenses; the assertion message says so. Tiny CRM bug 003 has a twist: stamping the close date is not enough — the month counter gives up on the whole list when any won lead has no date, and the seed has five; the fix is to skip them, not return zero. Anyone still on feature 1 does that first; feature 2 becomes homework.

YOUR TURN · 30 MIN

One bug on `main`. One feature on a branch.

SPLITIT

Bug 2, then feature 2
Removing a member crashes the page.
Then expense categories.

TINY CRM

Bug 3, then feature 2
"Won this month" is always zero. Then
conversion by source.

Files, test names and the ask: Week 4 page.

Five things you will see.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

2026-09-20

Rules, judge, loop

└ Five things you will see.

WHEN THE LOOP GOES WRONG

Five things you will see.

- 1 It edited a test. Discard. Say the rule. Run again.
- 2 "Done" — tests red. Ask for the output.
- 3 Same failure, three times. Stop. Read it yourself. New thread.
- 4 It fixed three other things. Discard. One task per loop.
- 5 Green, app wrong. Write down what the judge missed.

Every line here is a candidate rule for your `AGENTS.md`.

What did it get wrong? Write the rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md` with the reason.

That is how a rulebook grows: from what went wrong, one line at a time.

2026-09-20

Rules, judge, loop

└─What did it get wrong? Write the rule.

Collect the rules on the board; the good ones become the course's AGENTS.md for Block 2. Expect: "refresh the data after changing the seed files", "don't edit live data by hand", "show me the diff before committing". Reject vague ones ("be careful") — ask "how would you know it followed this?".

RETRO · 6 MIN

What did it get wrong? Write the rule.

- The worst thing it did today — one sentence.
- Turn it into a rule you could check.
- Commit it to `AGENTS.md`, with the reason.

That is how a rulebook grows: from what went wrong, one line at a time.

Next week you build alone.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`
- Push before you close the laptop.

Rulebook · judge · loop.

2026-09-20

Rules, judge, loop

└─ Next week you build alone.

Nothing to hand in. Next week's check-in: feature 2 merged, and a commit on AGENTS.md. Part II of the standards — walking away: scripted loops, goals, a second agent reviewing — comes with the team project in Block 3.

BEFORE NEXT WEEK

Next week you build alone.

- Feature 2: green, merged, pushed.
- One new rule in `AGENTS.md`.
- Push before you close the laptop.

Rulebook · judge · loop.